

Lecture 21: Performance portability

Anders Johansson
ajohans@sandia.gov

CS205 4/14/2026



Sandia
National
Laboratories



Harvard John A. Paulson
School of Engineering
and Applied Sciences

Overview

So far:

- Learned about CPUs with shared and distributed parallelism
- Performance models

Previously: First look at GPUs

- How to think of them
- Performance characteristics
- Intro to CUDA for NVIDIA GPUs

Today: Performance portability

- Why and how
- Introduction to Kokkos
- Notebook: <https://bit.ly/cs205lec21>

(see top500, LAMMPS-KOKKOS paper)

See notebook.

OpenMP vs. CUDA

- Similar concepts: Parallelism, thread assignment
- Only for CUDA: Extra memory allocation and movement

What about performance? (notebook)

OpenMP vs. CUDA

- Similar concepts: Parallelism, thread assignment
- Only for CUDA: Extra memory allocation and movement

What about performance? (notebook)

- Different data access patterns needed
- CPU: Per-core cache
- GPU: Per-warp cache

What do we need for performance portability?

Data

- Allocation
- Movement
- Synchronization?
- Layout flexibility

Execution

- Optimal thread mapping
- Multi-dimensional ranges
- Execution control
- Shared memory

Performance portability alternatives: Python

Numba CUDA

- Similar syntax, but slightly more convenient

CuPy: NumPy for GPUs

- Drop-in replacement
- GPU-accelerated whole-array operations
- Some support for kernels

PyTorch/Jax

- Quite similar syntax to NumPy
- GPU-accelerated whole-array operations
- Mostly supports different vendors + CPU

OpenMP for GPUs

OpenMP 4.5 (GCC/Clang):

#pragma omp target teams distribute parallel for simd

Can also manually split into different levels (teams/distribute/simd)

OpenMP 5.0 (NVHPC):

#pragma omp loop

Use at all levels, compiler will figure out optimal mapping.

Drawback: Very messy compiler support, little documentation.

C++ standard library

Most things in the algorithm header can be run in parallel (compiler-dependent)

NVIDIA: Can run on GPU using unified memory with NVHPC

```
void saxpy(float* x, float* y, int N, float a) {  
    std::transform(std::execution::par_unseq, x, x + N, y, y,  
                   [=](float xi, float yi){ return a * xi + yi; });  
}
```

Intel: Similar, but with a OneAPI execution policy

AMD: Working on it

Drawback: Messy compiler support, not standard C++.

Performance portable libraries that let you write and run kernels.

Libraries $\implies$ less compiler dependent

Kokkos from Sandia, RAJA from Lawrence Livermore

```
Kokkos::View<float*> x(N), y(N);  
Kokkos::parallel_for(N, KOKKOS_LAMBDA (int i) {  
    y(i) = a*x(i) + y(i);  
})
```

We will be using Kokkos because I like/know it.

See https://sc23.supercomputing.org/proceedings/tech_poster/poster_files/rpost217s3-file2.pdf

What is Kokkos?

A library that provides what we need to write portable parallel code

Data structures and parallel launch policies, KokkosKernels = BLAS etc.

At compile time, it translates the expressed parallelism to
CUDA/OpenMP/HIP/SYCL/etc.

```
cmake .. -DKokkos_ENABLE_CUDA=ON (or -DKokkos_ENABLE_OPENMP=ON)
```

Kokkos provides Views, which are multidimensional arrays (finally!)

```
Kokkos::View<float*> x(N);  
Kokkos::View<float**> y("y", M, N);
```

Can take lots of template arguments, see docs at
<https://kokkos.org/kokkos-core-wiki/>

Memory layout: LayoutRight (row-major), LayoutLeft (column-major)
If unspecified: LayoutRight on CPU, LayoutLeft on GPU

Memory space: CUDA, Host, HIP, etc. Where the data lives and can be accessed.
If unspecified: “Best” execution space for which the code was compiled (CUDA > OpenMP)

Moving data to the GPU

Typically have some data already on the CPU in a non-Kokkos data structure

```
std::vector<float> x_vec(N) = ...;
Kokkos::View<float*> x(N);
auto x_mirror = Kokkos::create_mirror_view(x);

for (int i = 0; i < N; i++) {
    x_mirror(i) = x_vec[i];
}

Kokkos::deep_copy(x, x_mirror);

where x_mirror is of type Kokkos::View<float*, Kokkos::HostSpace>
```

Launching kernels

```
Kokkos::parallel_for(N, KOKKOS_LAMBDA (int i) {  
    y(i) = a*x(i) + y(i);  
});
```

is short for

```
Kokkos::parallel_for(Kokkos::RangePolicy<>(0, N),  
    KOKKOS_LAMBDA (int i) {...});
```

which is short for

```
Kokkos::parallel_for(  
    Kokkos::RangePolicy<Kokkos::DefaultExecutionSpace>(0, N),  
    KOKKOS_LAMBDA (int i) {...});
```

Building example

Use `parallel_reduce`, and the lambda takes an extra argument

```
float sum;  
Kokkos::parallel_reduce(N, KOKKOS_LAMBDA(int i, float &ampmysum)  
    {...}, sum);
```

Default reduction is a sum, others exist: [https:](https://kokkos.org/kokkos-core-wiki/API/core/builtin_reducers.html)

[//kokkos.org/kokkos-core-wiki/API/core/builtin_reducers.html](https://kokkos.org/kokkos-core-wiki/API/core/builtin_reducers.html)

You can also do multiple reductions

See colab

Multidimensional loops

Replace the implicit `RangePolicy` with a `MDRangePolicy<Rank<n>>`

```
Kokkos::parallel_for("name",  
    Kokkos::MDRangePolicy<Kokkos::Rank<3>>({0,0,0}, {n1,n2,n3}),  
    KOKKOS_LAMBDA (int i, int j, int k) {...}  
);
```

Colab example

Hierarchical parallelism

`RangePolicy` and `MDRangePolicy` hide the details of thread blocks and warps

Sometimes, being aware of them can allow for optimizations

Examples: Use of shared memory, per-warp or per-block reductions

Kokkos terminology:

Kokkos	OpenMP	CUDA
Team	Thread	Thread block
Thread	-/thread	Warp
Vector	SIMD	Thread

Parallelization levels:

- `TeamThreadRange`: over warps
- `ThreadVectorRange`: over threads within warp
- `TeamVectorRange`: over warps and threads

Hierarchical parallelism: syntax

```
Kokkos::parallel_for("test",  
    Kokkos::TeamPolicy<>(N, nthreads, nvector),  
    KOKKOS_LAMBDA(Kokkos::TeamPolicy<>::member_type team) {  
        int teamid = team.league_rank(); // normally only this on  
        int nteams = team.league_size();  
        int teamsize = team.team_size();  
        int threadid = team.team_rank(); // "thread" ~= "warp"  
  
        ...  
    });
```

`nvector` is either an integer or omitted (1)

`nthreads` is an integer or `Kokkos::AUTO()`

Hierarchical thread mapping

Within a parallel for over a `TeamPolicy`, launch parallel loops over inner ranges:

```
Kokkos::parallel_for("test",
    Kokkos::TeamPolicy<>(N, nthreads, nvector),
    KOKKOS_LAMBDA(Kokkos::TeamPolicy<>::member_type team) {
        int i = team.league_rank();

        Kokkos::parallel_for(
            Kokkos::TeamVectorRange(team, n),
            [&] (int j) {
                ...
            }
        );
    });
```

Hierarchical thread mapping

Within a parallel for over a `TeamPolicy`, launch parallel loops over inner ranges:

```
Kokkos::parallel_for("test",
    Kokkos::TeamPolicy<>(N, nthreads, nvector),
    KOKKOS_LAMBDA(Kokkos::TeamPolicy<>::member_type team) {
        int i = team.league_rank();
        Kokkos::parallel_for(
            Kokkos::TeamThreadRange(team, n),
            [&] (int j) {
                Kokkos::parallel_for(
                    Kokkos::ThreadVectorRange(team, n),
                    [&] (int k) {...}
                );
            }
        );
    }
);
```

Do operation only once per team:

```
Kokkos::single(Kokkos::PerTeam(team), [&] () {...});
```

Team-wide barrier:

```
team.team_barrier();
```


See colab

Shared memory

CUDA and Kokkos offer per-block/per-warp shared (“scratch”) memory

Ask for shared memory when creating the policy:

```
Kokkos::TeamPolicy<>(...).set_scratch_size(  
    0, Kokkos::PerTeam(size)  
)
```

Level 0 = fastest, level 1 = usually normal memory

Can be either per team or per thread (or both)

See <https://kokkos.org/kokkos-core-wiki/API/core/policies/TeamPolicy.html>

Getting shared memory

Inside the kernel:

```
ScratchVec x(team.team_scratch(0), N);
```

Views in the shared memory:

```
typedef Kokkos::View<  
    float*,  
    typename Kokkos::DefaultExecutionSpace::scratch_memory_space  
> ScratchVec;
```

This can be used to calculate required size:

```
int shmem_size = ScratchVec::shmem_size(N);
```

Summary

Simple 1D and multi-D loops in Kokkos are easy

Hierarchical parallelism offers more control

Shared memory can help if a team reads the same memory multiple times

Further reading: <https://kokkos.org/kokkos-core-wiki/>

- Programming guide, chapters 1–8
- Lectures 1–4.2